

Feature Implementation Plan

Prepared: July 2026 | Version 1.0 | Status: Draft

Feature	Budget Stretch — borrow budget between categories or months
Phases	6 implementation phases
Scope	DB schema · API · OpenAPI codegen · 4 frontend tabs
New DB tables	budget_stretches
Modified files	~18 files (schema, routes, spec, 4 pages, 2 chart components)
Icon	ArrowRightLeft (Lucide) — conveys transfer between two sides

What is Budget Stretch?

Budget Stretch lets a user temporarily expand a category's monthly budget in one of two ways, without permanently changing the category's base budget setting. Each stretch is always tied to exactly one transaction.

Type	Description	Constraint
Cross-Category	Transfer budget from category B to category A within the same month. Category A's effective budget grows; B's shrinks.	Both categories belong to the same user. One stretch per transaction.
Cross-Month	Borrow from the same category's next month. This month's effective budget grows up to 150%; next month's shrinks to 50%.	Max 50% of category budget. Allowed once per two consecutive months. If stretched in March, April is locked; next allowed: May.

- A transaction may carry **at most one** stretch — it cannot simultaneously be stretched cross-category and cross-month.
- Cross-month stretches require the amount to be $\leq$ **50% of the target category's base budget**. Accordingly, this month's effective budget is base + stretch (max 150%), next month's is base – stretch (min 50%).
- The two-month cooldown for cross-month stretches is per-category: if category A was stretched in March→April, it cannot be cross-month-stretched again until May→June.
- Cross-category stretches do **not** affect the total monthly budget — they redistribute it.
- Cross-month stretches **do** change the effective total monthly budget for both months.
- If a user has **no categories**, the Stretch section is hidden in the transaction form.

Where Stretch Appears in the UI

Tab	What changes	Colour signal
Home	Transaction rows: orange 'Stretch' badge Monthly total: adjusted + orange label beneath	Orange badge, orange text
Categories	Category cards: 'stretched by ±X' orange label (cross-cat: both donor and receiver labeled; cross-month: receiver in current month + hardcoded deduction in ne	Orange font labels
Dashboard	DonutBudgetChart: orange border on stretched segments Budget total adjusted for cross-month stretches Donut 360° represents adjusted budget when any cross-month stretch is active	Orange segment border (overridden by red if also
Household	Household donut ring: orange border if user has any cross-month stretch Personal donut: same rules as Dashboard donut	Orange ring border on user slice Orange ring

Phase

Database Schema & Migration

New table · Drizzle schema · Migration file

Small

budget_stretches

Create `lib/db/src/schema/budget_stretches.ts` with the following columns:

Column	Type	Description
id	serial PK	Auto-increment primary key
userId	integer FK → users	Owner of the stretch
transactionId	integer FK → transactions	The one transaction this stretch belongs to (unique)
month	text (YYYY-MM)	The month of the stretched category (current month)
toCategoryId	integer FK → categories	Category receiving the extra budget
fromCategoryId	integer FK → categories	Category donating budget (same as toCategoryId for cross-month)
amount	numeric(12,2)	Amount of budget transferred
stretchType	text	'cross_category' 'cross_month'
createdAt	timestamp	Record creation timestamp

Constraints & Indexes

- **UNIQUE** constraint on `transactionId` — one stretch per transaction, enforced at DB level.
- Index on `(userId, month)` — fast lookup for summary endpoints.
- Index on `(toCategoryId, month)` — used by cross-month cooldown check.
- No FK constraint on `transactionId` to avoid circular imports (same pattern as `splitId` on transactions).

Files to create / modify:

- **Create:** `lib/db/src/schema/budget_stretches.ts`
- **Modify:** `lib/db/src/schema/index.ts` — add export
- **Run:** `pnpm --filter @workspace/db run generate` → produces migration `0007_budget_stretches.sql`
- Migration applied automatically at API server startup via `migrate()`.

[artifacts/api-server/src/routes/budget-stretches.ts](#)

Method + Path	Description
GET /budget-stretches?month=YYYY-MM	List all stretches for the authenticated user in a given month. Used by summary endpoints
POST /budget-stretches	Create a stretch. Accepts: transactionId, toCategoryId, fromCategoryId, amount, stretchT
DELETE /budget-stretches/:id	Remove a stretch. Only the owning user may delete. Invalidates nothing server-side — th

Validation Logic (POST /budget-stretches)

Rule 1 – One stretch per transaction: Query `budget_stretches` WHERE `transactionId = X`. If found, reject with 409.

Rule 2 – Category ownership: Both `toCategoryId` and `fromCategoryId` must belong to the authenticated user. Reject with 403 otherwise.

Rule 3 – Cross-month amount cap: When `stretchType = 'cross_month'` (`fromCategoryId === toCategoryId`), the amount must be $\leq 50\%$ of the category's base budget. Reject with 422 with a message showing the max allowed.

Rule 4 – Cross-month cooldown: For `cross_month`, check if any stretch with `toCategoryId = X` and `month = previous YYYY-MM` exists. If yes, reject with 422: 'Cross-month stretch locked for this category until [next available month].'

Rule 5 – Transaction exists & belongs to user: Validate `transactionId` ownership.

Rule 6 – Month derivation: The month is derived from the transaction's date field (first 7 chars), never passed in by the client, to prevent spoofing.

Route Registration

- Modify: [artifacts/api-server/src/routes/index.ts](#) — mount `budgetStretchesRouter` at `/api`.

lib/api-spec/openapi.yaml

Add a `budgetStretches` tag and the following paths:

New Paths

- `GET /budget-stretches` (operationId: `listBudgetStretches`) — query param: `month` (YYYY-MM)
- `POST /budget-stretches` (operationId: `createBudgetStretch`)
- `DELETE /budget-stretches/{id}` (operationId: `deleteBudgetStretch`)

New Schemas

- `BudgetStretch` — `id`, `transactionId`, `userId`, `month`, `toCategoryId`, `fromCategoryId`, `amount`, `stretchType`, `createdAt`
- `BudgetStretchInput` — `transactionId`, `toCategoryId`, `fromCategoryId`, `amount`, `stretchType`

Modify Existing Schemas

- `Transaction` schema: add optional `stretch` field (nullable `BudgetStretch` object, populated server-side when the transaction has an associated stretch).
- `CategorySpending` schema (used by `GET /summary/spending`): add `stretchAmount` (number, default 0), `stretchType` (string | null), `isStretched` (boolean).
- `SpendingSummary` response: add `adjustedTotalBudget` (number | null) — set when any cross-month stretch is active.

Codegen

- After spec changes, run: `pnpm --filter @workspace/api-spec run codegen`
- This regenerates `lib/api-client-react/src/generated/api.ts` and `lib/api-zod/src/generated/` — do not manually edit these files.

Phase 4

API – Stretch-Aware Summary Endpoint

Modify GET /summary/spending · Adjust budgets · Inject stretch metadata

Medium

artifacts/api-server/src/routes/summary.ts

The `getSpendingGrouped()` function must be extended to fetch and apply stretches when computing effective budgets for each category.

Logic Changes

- 1. Fetch stretches for the month:** At the start of `getSpendingGrouped()`, load all `budget_stretches` WHERE `userId = X` AND `month = YYYY-MM`.
- 2. Build a stretch map:** Key = `categoryId` → { `effectiveBudget`, `stretchAmount`, `stretchType`, `isStretched` }.
- 3. Apply cross-category stretches:** `toCategoryId` gets `base_budget + amount`; `fromCategoryId` gets `base_budget - amount`. Total budget unchanged.
- 4. Apply cross-month stretches:** `toCategoryId` (in current month) gets `base_budget + amount`. Total budget for this month increases by the sum of all cross-month stretch amounts.
- 5. Previous month deduction:** For the previous month's query, any cross-month stretches where `fromCategoryId = toCategoryId` and `month = current-1` reduce that category's effective budget by the amount.
- 6. Return adjustedTotalBudget:** If any cross-month stretch is active, set `adjustedTotalBudget` on the response; otherwise null.
- 7. Enrich each category item** with: `isStretched`, `stretchAmount`, `stretchType` from the stretch map.

artifacts/api-server/src/routes/transactions.ts

The `enrichTransaction()` function and the GET /transactions route must join `budget_stretches` and return the `stretch` field on each transaction that has one.

- Batch-load stretches for all returned transaction IDs — never N+1 queries.

artifacts/finance-app/src/pages/Transactions.tsx

The Stretch section is added to `TxForm` (both create and edit), positioned immediately below the Goal contribution section — matching its layout pattern.

UI Structure

- **Section header:** 'Stretch budget' label with `ArrowRightLeft` icon (Lucide). Only rendered when the user has at least one category.
- **Three-way toggle** (matches Goal toggle style): **No** (default) · **Fully** · **Partially**.
- When **Fully** or **Partially** selected, two dropdowns appear:
 - **'Which'** dropdown: target category (`toCategoryId`). Pre-selects the transaction's current category if set.
 - **'From which'** dropdown: source category (`fromCategoryId`). Top item by default: same category as 'Which' with label '(from next month)'. Below it: all other user categories.
- When **Partially** selected: numeric input for the stretch amount appears below the dropdowns.

Validation UX

Cross-month path (user selects same category in both dropdowns):

- If transaction amount > 50% of that category's base budget: show inline warning 'Cross-month stretch not available — transaction exceeds 50% of this category's budget.' The 'from next month' option is hidden from the second dropdown for this transaction.
- Partial amount input: show 'Max: [50% of budget] [currency]'. If entered amount exceeds 50% of budget: disable Submit button, show 'Amount too large — maximum allowed is [X] ([currency]).'

Cross-category path (different categories in both dropdowns):

- No special amount cap, but submit is disabled if amount field is empty (Partially mode).

Form Submission

- On transaction save (POST or PATCH), if stretch toggle ≠ 'No':
 - After the transaction is saved and its ID is available, call `POST /budget-stretches` with the stretch payload.
 - If the transaction already has a stretch (edit mode) and the user changes or removes it: call `DELETE /budget-stretches/:id` first, then create the new one.
 - Invalidate `getListTransactionsQueryKey()` and `getGetSpendingSummaryQueryKey()` after stretch operations.

HomeSpending.tsx

Transaction rows: if `transaction.stretch` is present, render an orange 'Stretch' badge (`bg-orange-500/15 text-orange-400 text-[10px] font-semibold`) to the right of the category badge on that row.

Monthly total budget display:

- When `adjustedTotalBudget` is returned by GET `/summary/spending` (non-null, $\neq$ base total): display the adjusted total in place of the base total.
- Render an orange label underneath: `'+X [currency] stretched from next month'` (current month) or `'-X [currency] stretched to last month'` (following month showing deduction).

Categories.tsx

Fetch GET `/budget-stretches?month=YYYY-MM` alongside existing category data.

For each category card:

- **Cross-category stretch (receiver):** show orange label 'stretched by +X [currency]'.
- **Cross-category stretch (donor):** show orange label 'stretched by -X [currency]'.
- **Cross-month stretch (current month — receiver):** show orange label 'stretched by +X [currency] from next month'.
- **Cross-month stretch (next month — deduction):** show hardcoded orange label 'stretched by -X [currency] from last month'. This requires fetching the previous month's cross-month stretches for this category.

Labels are shown in orange (`text-orange-400`) beneath the existing budget/spending info inside the category card.

Dashboard.tsx DonutBudgetChart.tsx

Changes to DonutBudgetChart:

- Extend `SpendingItem` type with: `isStretched?: boolean`, `stretchAmount?: number`, `stretchType?: string`.
- Add new `GroupBorder` rendering path: when `isStretched === true`, draw an orange outer stroke ring on the segment (same technique as the existing red ring for over-budget). Orange ring is suppressed if the segment is also over-budget (red ring takes priority). Both labels still appear (red % overflown + orange stretch info).
- Accept an optional `adjustedTotalBudget` prop. When provided, the 360° of the donut represent `adjustedTotalBudget` instead of the sum of all category budgets. This prop is passed from Dashboard when any cross-month stretch is active for the viewed month.

Changes to Dashboard.tsx:

- Read `adjustedTotalBudget` from the GET `/summary/spending` response.
- Pass it to `DonutBudgetChart` as a prop when non-null.
- Show adjusted budget total in the stat cards with the same orange label logic as Home tab.

Household.tsx HouseholdDonutChart.tsx

Changes to HouseholdDonutChart:

Phase Dependency Map

Phase	Depends on	Can run in parallel with
Phase 1 – DB Schema	—	Nothing (must be first)
Phase 2 – API CRUD	Phase 1 (schema types)	Phase 3 (spec can be written independently)
Phase 3 – OpenAPI + Codegen	Phase 1 (schema knowledge)	Phase 2 (no code dependency)
Phase 4 – Summary endpoint	Phases 1 + 2	Phase 3 codegen (spec changes inform, but not block)
Phase 5 – Tx Form UI	Phase 3 codegen complete	Phase 4 (independent of summary changes)
Phase 6 – All 4 Tabs	Phases 3 + 4 complete	Can start Phase 6 Home+Categories while Dashboard awaits Phase 4

- 1. Phase 1 (DB) → then fork:
 - a) Phase 2 (API CRUD) & Phase 3 (OpenAPI) in parallel
- 2. Run codegen once Phase 3 spec is finalised
- 3. Phase 4 (summary endpoint) — requires Phase 1 + 2
- 4. Phase 5 (transaction form) — requires codegen from Phase 3
- 5. Phase 6 (four tabs) — Home + Categories can start after Phase 3 codegen; Dashboard + Household wait for Phase 4

Files to Create or Modify

File	Action	Phase
lib/db/src/schema/budget_stretches.ts	Create	1
lib/db/src/schema/index.ts	Modify	1
lib/db/migrations/0007_budget_stretches.sql	Generate	1
lib/api-spec/openapi.yaml	Modify	3
lib/api-client-react/src/generated/api.ts	Regen	3
lib/api-zod/src/generated/ (all files)	Regen	3
artifacts/api-server/src/routes/budget-stretches.ts	Create	2
artifacts/api-server/src/routes/index.ts	Modify	2
artifacts/api-server/src/routes/transactions.ts	Modify	4
artifacts/api-server/src/routes/summary.ts	Modify	4
artifacts/finance-app/src/pages/Transactions.tsx	Modify	5
artifacts/finance-app/src/pages/HomeSpending.tsx	Modify	6
artifacts/finance-app/src/pages/Categories.tsx	Modify	6
artifacts/finance-app/src/pages/Dashboard.tsx	Modify	6
artifacts/finance-app/src/pages/Household.tsx	Modify	6
artifacts/finance-app/src/components/DonutBudgetChart.tsx	Modify	6
artifacts/finance-app/src/components/HouseholdDonutChart.tsx	Modify	6

EDGE CASES & IMPLEMENTATION NOTES

Key Edge Cases

- **Deleting a stretched transaction:** The stretch row should be cascade-deleted via application logic (or a DB-level ON DELETE CASCADE on budget_stretches.transactionId). The effective budget in both affected months automatically reverts to normal on the next summary fetch since the stretch row is gone.
- **Editing a transaction's category after setting a stretch:** If the user changes the transaction's category in edit mode, the stretch's toCategoryId should be updated accordingly, or the stretch should be cleared and re-created. Simplest: clear the stretch and prompt the user to re-set it.
- **Currency:** Stretch amounts are stored in the user's native currency. No conversion is needed — the category budget is always in native currency.
- **Household context:** Stretches are personal (userId-scoped). A category shared to a household goal does not propagate its stretch to other members.
- **Cross-month cooldown when a stretch is deleted:** Deleting a cross-month stretch lifts the cooldown — the category becomes stretchable again in the freed month pair.

- **Recurring payments:** The stretch UI is shown for recurring-payment transactions the same as for regular transactions — no special handling needed.
- **Donut total budget = 0:** If all categories have no budget set, `adjustedTotalBudget` remains null and the donut renders normally.

Icon selection: `ArrowRightLeft` (Lucide React) — already installed in the project. Conveys bilateral transfer between two sides, fitting both stretch modes. Used in the `TxFom` section header and in any stretch badge/label where an icon is appropriate.